

Para este módulo utilizaremos el paquete oficial **connectivity_plus**, que actúa como un observador reactivo sobre los demonios de red nativos de Android (**ConnectivityManager**) e iOS (**NWPathMonitor**).

Tutorial: Sensores de Entorno y Monitoreo Reactivo de Conectividad (Wi-Fi/Red)

Este módulo práctico enseña a los estudiantes a implementar arquitecturas móviles capaces de auditar y reaccionar en tiempo real al estado de la red del dispositivo, gestionando la asincronía para optimizar las peticiones de datos orientadas a APIs o bases de datos remotas.

1. Fundamentos Teóricos y Conceptos del Paquete **connectivity_plus**

En el desarrollo de software, el acceso a la red se trata como un **sensor de estado del entorno**.

- **ConnectivityResult (Enum de Estado):** Es el enumerador provisto por el paquete que mapea la interfaz física de red por la cual el dispositivo está enrutando paquetes de datos. Los estados principales son:
 - **ConnectivityResult.wifi:** El dispositivo está enlazado a una red local inalámbrica.
 - **ConnectivityResult.mobile:** El dispositivo depende de la red de datos celulares (3G/4G/5G).
 - **ConnectivityResult.ethernet:** El dispositivo está conectado vía cable (común en emuladores o smart TVs).
 - **ConnectivityResult.none:** El dispositivo se encuentra en aislamiento de red completo (Modo Avión o antenas apagadas).
- **Monitoreo Multinterfaz (Listas de Resultados):** Las versiones modernas del paquete retornan un **List<ConnectivityResult>** en lugar de un único valor. Esto responde a la arquitectura de hardware de los smartphones actuales, que permiten la **aceleración de red dual** (mantener tráfico simultáneo por Wi-Fi y datos móviles para mejorar la latencia).
- **Falsa Conectividad (El Dilema del Gateway):** **connectivity_plus** audita si la interfaz física de red está encendida y conectada a un router/antena, **pero no garantiza que dicho router tenga salida real a Internet**. Un teléfono puede estar conectado a un Wi-Fi escolar (estado: **.wifi**) pero bloqueado por el portal cautivo sin acceso real al backend.

2. Configuración del Entorno (Plataforma Nativa)

La auditoría básica del estado de la red no se considera una violación crítica de la privacidad en las versiones modernas de los sistemas operativos, pero es obligatorio declarar el permiso de hardware en Android.

En Android (**android/app/src/main/AndroidManifest.xml**)

Añade la siguiente línea de permiso de red justo antes de la etiqueta de apertura **<application>**:

```
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

📁 Estructura de Carpetas del Módulo

Siguiendo la arquitectura limpia por características definida en el proyecto, la carpeta queda organizada de la siguiente manera:

```
mi_app_sensores/  
├── pubspec.yaml  
└── lib/  
    ├── main.dart  
    ├── core/  
    │   └── theme/  
    │       theme.dart  
    └── features/  
        ├── home/  
        │   ├── screens/  
        │   └── home_screen.dart  
        └── wifi/                                <-- CARPETA DEL MÓDULO ACTUAL  
            ├── screens/  
            └── wifi_screen.dart
```

Asegúrate de tener agregada la dependencia en el *pubspec.yaml*:

```
dependencies:  
  flutter:  
    sdk: flutter  
  connectivity_plus: ^6.1.0
```

3. Código Fuente Completo y Comentado

Archivo *lib/features/wifi/screens/wifi_screen.dart*

```
import 'package:flutter/material.dart';  
import 'package:connectivity_plus/connectivity_plus.dart';  
import 'dart:async';  
  
class WifiScreen extends StatefulWidget {  
  const WifiScreen({super.key});
```

```

@override
State<WifiScreen> createState() => _WifiScreenState();
}

class _WifiScreenState extends State<WifiScreen> {
  String _instantStatus = "Desconocido. Presiona el botón.";
  String _liveStatus = "Escuchando cambios de red...";
  bool _isOffline = false;

  // Tubería de control para escuchar de manera asíncrona los cambios de hardware
  StreamSubscription<List<ConnectivityResult>>? _connectivitySubscription;
  final Connectivity _connectivity = Connectivity();

  @override
  void initState() {
    super.initState();
    // Iniciamos el monitoreo reactivo desde que la pantalla entra al árbol de
    widgets
    _initLiveTracking();
  }

  // 1. Verificación Instantánea (Paradigma Request-Response / Future)
  // Ideal para evaluar la red justo antes de disparar un commit a la Base de
  Datos
  Future<void> _checkInstantConnectivity() async {
    try {
      List<ConnectivityResult> results = await _connectivity.checkConnectivity();

      setState(() {
        _instantStatus = _parseResultToString(results);
      });
    } catch (e) {
      setState(() {
        _instantStatus = "Error de hardware: $e";
      });
    }
  }

  // 2. Monitoreo en Tiempo Real (Paradigma Reactivo / Streams)
  void _initLiveTracking() {
    // Escuchamos el canal de eventos del sistema operativo
    _connectivitySubscription =
    _connectivity.onConnectivityChanged.listen((List<ConnectivityResult> results) {
      setState(() {
        _liveStatus = _parseResultToString(results);

        // Evaluamos si el dispositivo está completamente aislado de la red
        _isOffline = results.isEmpty || results.contains(ConnectivityResult.none);
      });
    });
  }

  // Método de ingeniería auxiliar para parsear y traducir los enums de hardware
  String _parseResultToString(List<ConnectivityResult> results) {

```

```

    if (results.isEmpty || results.contains(ConnectivityResult.none)) {
      return "🔴 MÓDULO OFFLINE - Red Inaccesible";
    }

    List<String> activeInterfaces = [];

    if (results.contains(ConnectivityResult.wifi)) {
      activeInterfaces.add("🟢 Interfaz Wi-Fi Activa");
    }
    if (results.contains(ConnectivityResult.mobile)) {
      activeInterfaces.add("📶 Datos Móviles Activos");
    }
    if (results.contains(ConnectivityResult.ethernet)) {
      activeInterfaces.add("🔌 Enlace Ethernet Detectado");
    }

    return activeInterfaces.join(" & ");
  }

  @override
  void dispose() {
    // CRUCIAL: Romper la suscripción al flujo del S.O. al destruir el widget
    // Previene hilos huérfanos y fugas de memoria en segundo plano
    _connectivitySubscription?.cancel();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Sensor de Estado: Conectividad Wi-Fi')),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.stretch,
          children: [
            // Sección de Auditoría bajo demanda
            Card(
              elevation: 4,
              color: Colors.blue.withOpacity(0.05),
              child: Padding(
                padding: const EdgeInsets.all(16.0),
                child: Column(
                  children: [
                    const Text('Auditoría Síncrona Instantánea', style:
TextStyle(fontSize: 16, fontWeight: FontWeight.bold)),
                    const SizedBox(height: 12),
                    Text(_instantStatus, textAlign: TextAlign.center, style: const
TextStyle(fontFamily: 'monospace', fontSize: 14)),
                    const SizedBox(height: 12),
                    ElevatedButton.icon(
                      onPressed: _checkInstantConnectivity,
                      icon: const Icon(Icons.network_check),
                      label: const Text('Consultar Estado de Interfaces'),

```

```

        ),
      ],
    ),
  ),
  const SizedBox(height: 20),

  // Sección de Monitoreo Reactivo Continuo
  Card(
    elevation: 4,
    // Cambiamos el color de la UI dinámicamente si el sistema detecta
    que está offline
    color: _isOffline ? Colors.red.withOpacity(0.08) :
Colors.green.withOpacity(0.08),
    child: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Column(
        children: [
          const Text('Auditoría Asíncrona Reactiva (Stream)', style:
TextStyle(fontSize: 16, fontWeight: FontWeight.bold)),
          const SizedBox(height: 16),
          Container(
            width: double.infinity,
            padding: const EdgeInsets.all(12),
            decoration: BoxDecoration(
              color: Colors.white,
              borderRadius: BorderRadius.circular(8),
              border: Border.all(color: Colors.grey.shade300),
            ),
            child: Text(
              _liveStatus,
              textAlign: TextAlign.center,
              style: const TextStyle(fontFamily: 'monospace', fontSize:
15, fontWeight: FontWeight.bold),
            ),
          ),
          const SizedBox(height: 12),
          Text(
            _isOffline
              ? '⚠️ Alerta de Sistema: El tráfico de red está
bloqueado.'
              : '✅ Sistema en Línea: Listo para sincronizar datos.',
            style: TextStyle(fontSize: 12, color: _isOffline ?
Colors.red[900] : Colors.green[900], fontWeight: FontWeight.w500),
          ),
        ],
      ),
    ),
  ],
),
);
}

```

}